# Как собрать миллиардный датасет для редактирования изображений

Архипкин Владимир

# Постановка задачи

Учимся редактировать изображение по инструкции

(картинка 1, инструкция, картинка 2)

Картинка 1



Инструкция

**«Замени синюю футболку на красный свитер»**

Картинка 2



# Наивный подход: таксономия правок + синтетика

Для каждого типа правки выбираем свой способ генерации пар

Редактирование

## Замена объектов

Добавление и удаление

**Detector + inpainting**

Находим объект и маску,
генерируем замену



«Замени яблоко апельсином»

## Замена стиля

Цвет и материал

**StyleGAN + адаптация**

Один z у исходного
и стилевого генератора



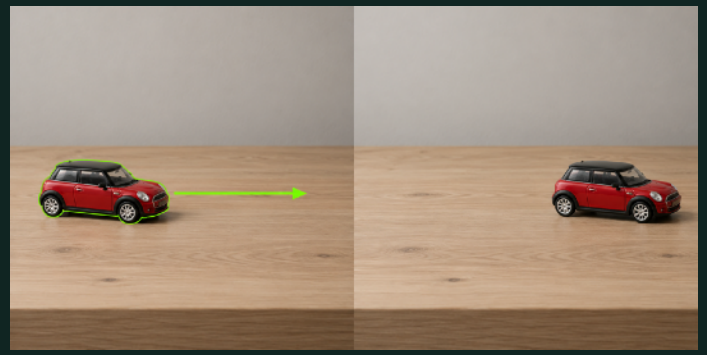
«Сделай акварельный портрет»

## Перенос объектов

Масштаб и поворот

**Segmentation +
DragonDiffusion**

Получаем маску сегментацией,
задаём вектор перемещения



«Перемести машинку вправо»

# Миллиард изображений ≈ 1T токенов

Для претрейна это всё ещё небольшой объём

**1 млрд** × **1024** ≈ **1T**

изображений | токена на изображение | визуальных токенов

Для 512×512: VAE сжимает в 8 раз, патч в латенте 2×2

Для масштаба, текстовый претрейн:

**DeepSeek-V3: 14,8T токенов**

**MAI-Thinking-1: 30T токенов (претрейн базы MAI-Base-1)**

Это сравнение числа токенов, а не вычислений или объёма информации

Один проход по отдельным изображениям. Пары и референсы считаются отдельно

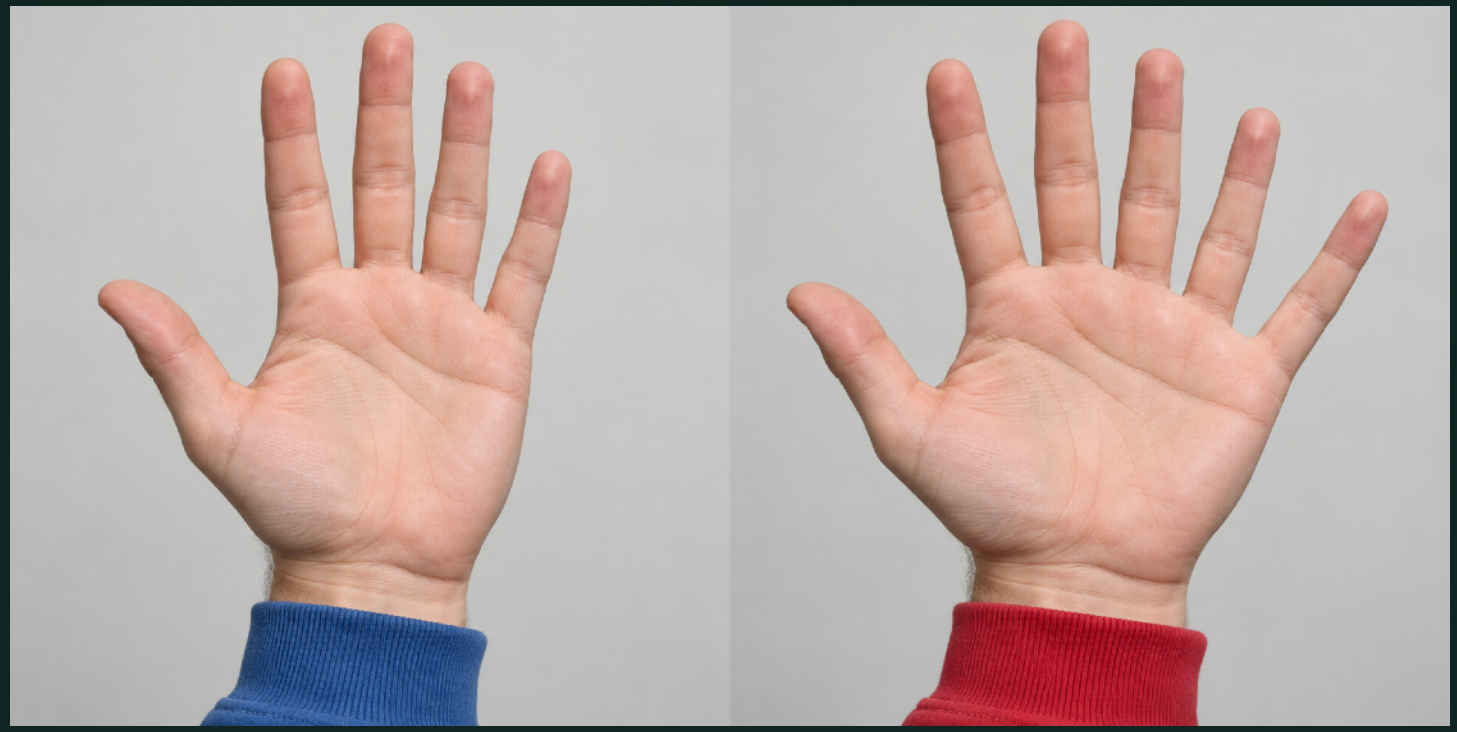
# Артефакты становятся массовой проблемой

Даже большие диффузионные модели дают некорректные пары

Инструкция: «Сделай рукав красным»

До

После: лишний палец



**≈1% брака**

**Модели уровня GPT Image 2.5**

Очень дорогая генерация

10 млн плохих пар на 1 млрд

**до ≈90%**

**Небольшие диффузионные модели**

До 900 млн плохих пар на 1 млрд

Оценки лектора зависят от задачи и критериев отбора. Пример ошибки сгенерирован

# Генерация миллиарда пар стоит дорого

1 млрд синтетических пар

≈100 000

**GPU-дней**

При среднем бюджете
8,64 GPU·с на пару

Идеальное распараллеливание

**1000 GPU**

100 дней

**100 GPU**

1000 дней

Отбраковка дополнительно повышает стоимость полезной пары

Оценка порядка величины. Скорость зависит от модели, GPU и пайплайна

# Таксономия плохо масштабируется

## Ручной список не охватывает пространство запросов

Огромное число сценариев и сочетаний остаётся за рамками таксономии

## На каждый тип правки нужен отдельный пайплайн

Растут затраты на ML-инженеров, отладку и поддержку

## Референсы и комбинации правок резко усложняют задачу

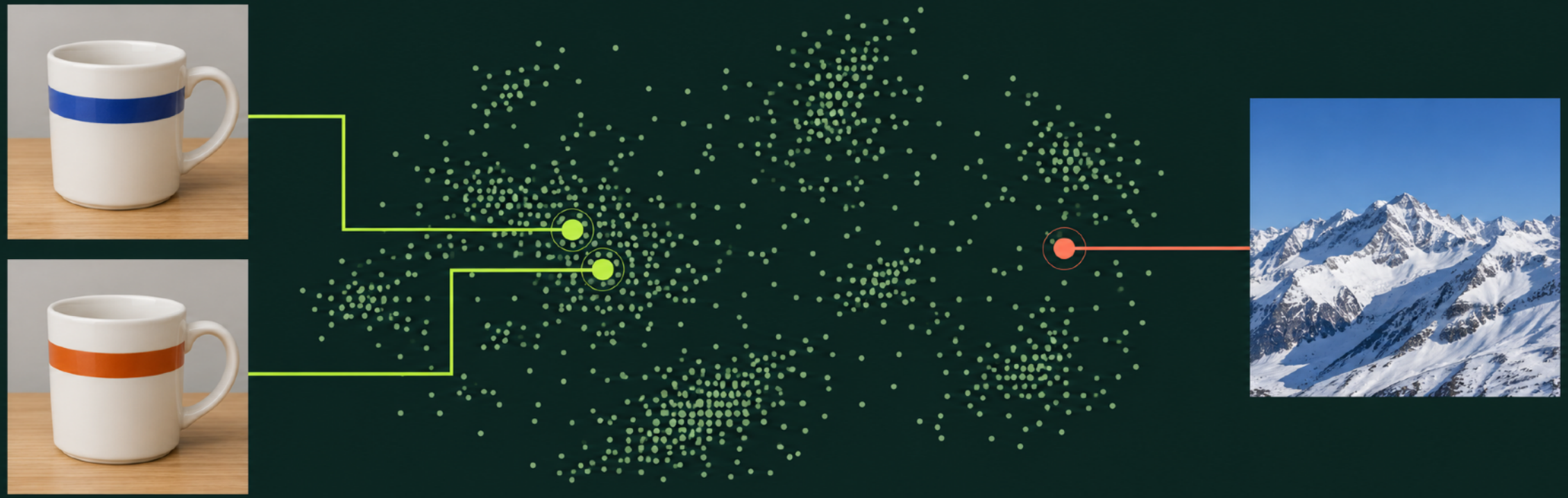
Несколько референсов и сложные сценарии почти невозможно покрыть

## Плюс: прототип можно отлаживать на 1 GPU

и прокачать свои навыки ML-инженерии

# Майнинг похожих изображений

Ищем соседей в большом корпусе неразмеченных картинок



**Соседние точки: похожие картинки**

**Далёкая точка: другой сюжет**

Косинусное расстояние между эмбеддингами:

$$d = 1 - \cos(z_1, z_2)$$

Каждая точка соответствует изображению. Показана схематичная 2D-проекция

# Алгоритм поиска похожих изображений

## 1

### Дедубликация

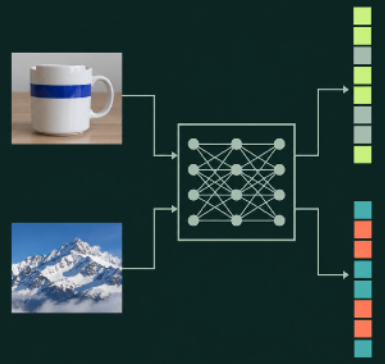
Удаляем точные дубли
Сохраняем различия



## 2

### Эмбеддинги

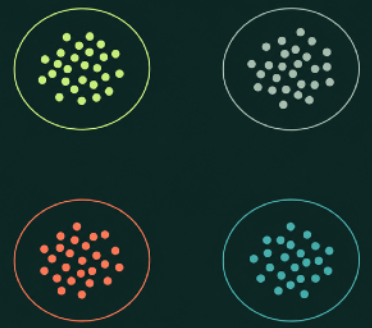
Кодируем картинки
CLIP, SigLIP и аналоги



## 3

### Кластеризация

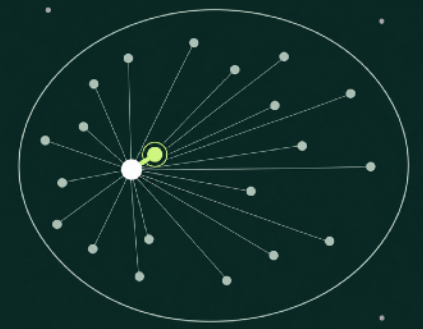
Группируем близкие
эмбеддинги



## 4

### Ближайший сосед

Сравниваем со всеми
векторами в кластере



**Все пары: $O(n^2d)$. K-means: $O(nkdT)$. В кластерах: $O(d \cdot \Sigma m_k^2)$**

Пример: $n=10^9$, $k=10^4$, $d=768$, $T=20$. Равномерные кластеры по $10^5$ векторов

**Без кластеров: ≈$1{,}5\times10^{21}$ FLOP**

**K-means + поиск: ≈$4{,}6\times10^{17}$ FLOP**

K-means: $3{,}1\times10^{17}$ + поиск: $1{,}5\times10^{17}$ FLOP. ≈3300× меньше арифметики. Оценка без I/O

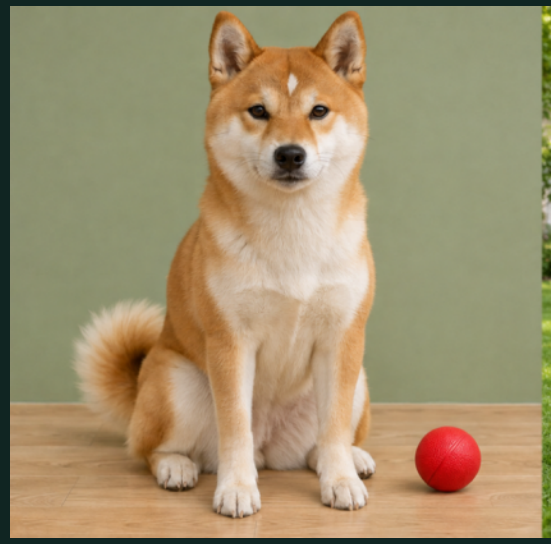
# Ошибки поиска: подмена объекта

Косинусная близость не гарантирует сохранения объекта

Правка: «Перенеси собаку на лужайку у дома и сделай мяч синим»

## Корректный матч

Исходное



Найденное



Меняются фон и цвет мяча
Собака сохраняется

## Некорректный матч

Исходное



Найденное



Вместе с фоном меняется собака
Пара нарушает инструкцию

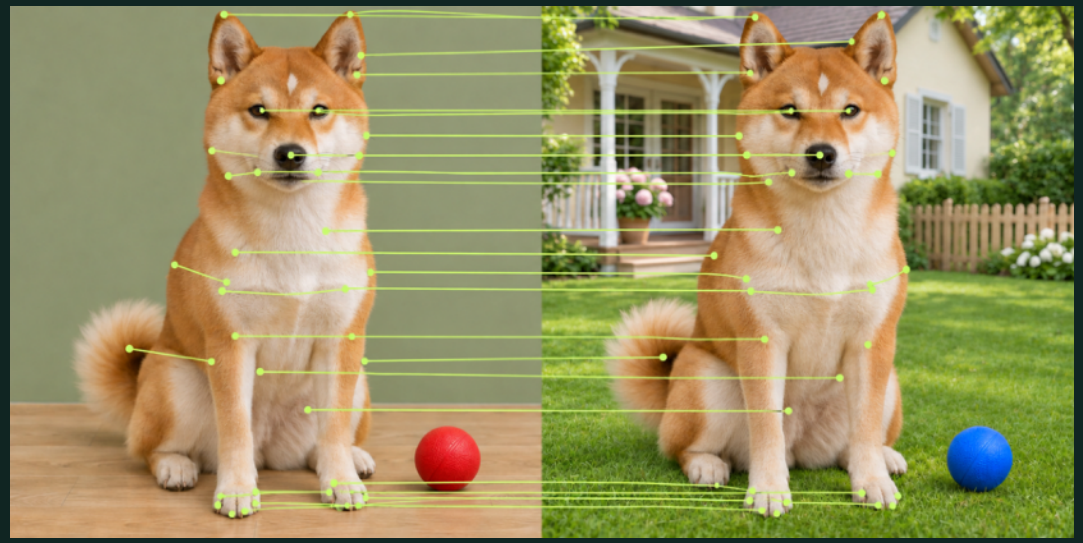
Схематический пример. Значения cos sim не измерялись

# Матчер

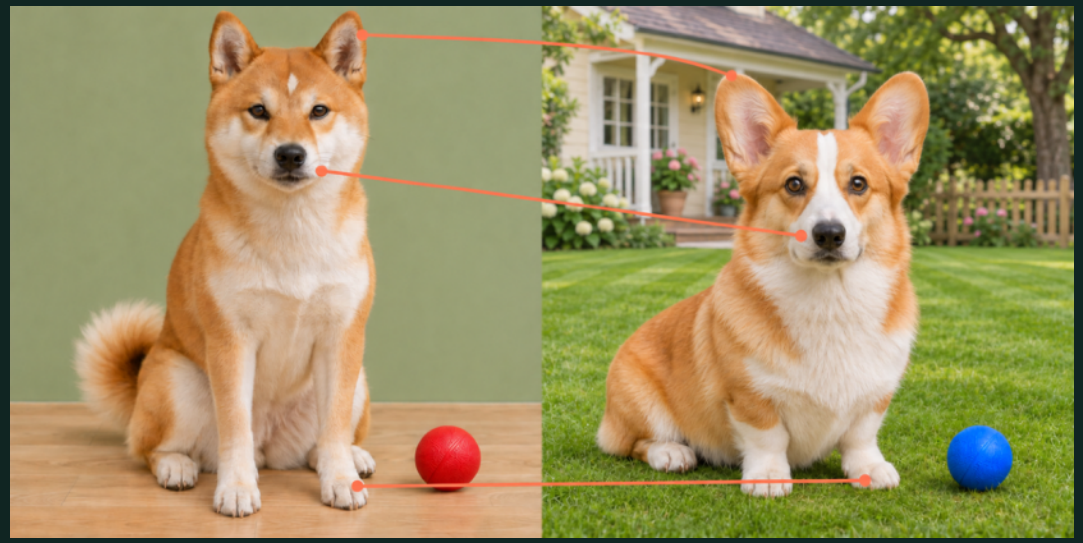
После поиска по эмбеддингам проверяем пары с помощью LoFTR

Модель ищет локальные соответствия. N обозначает их число

## Много соответствий



## Мало соответствий



**N ≥ т: сохраняем пару**

Скорее всего, объект тот же

**N < т: отбрасываем пару**

Скорее всего, объект другой

**Порог т подбираем на отложенной размеченной выборке**

Схема соответствий. Общий фон может давать ложные совпадения

# Мультиреференсное редактирование

Поиск и матчер работают отдельно для каждого объекта

## 1. Детектор

Находим объекты на таргете
и выделяем отдельные кропы

## 2. Поиск по кропам

Эмбеддинги, кластеры,
ближайшие кропы в корпусе

## 3. Матчер

Каждую найденную пару кропов
проверяем с помощью LoFTR

Схема, не результат запуска моделей



**Результат: таргет + референсы к разным объектам**

# Эмбеддер под нужную характеристику

Выбираем, по какому признаку искать близкие примеры

## Стиль изображения

Для задачи замены стиля



## Движение в видео

Для задачи переноса действия




**Меняем эмбеддер и пересчитываем индекс. Схема поиска та же**

# Кэпшенинг

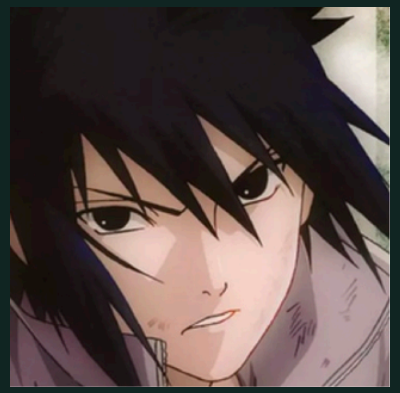
Найденной паре нужна инструкция: что и как изменить

## Запрос к VLM

Максимально подробно опиши все изменения от первой картинки ко второй

Без пропусков и выдумок

Исходное



Целевое



## Фрагмент подробной инструкции

Преобразуй цветной аниме-портрет Саске в чёрно-белую мангу. Убери цвет и мягкие градиенты. Используй белую бумагу, чёрную тушь и штриховку. Сделай волосы почти сплошной чёрной массой с острыми прядями.

Поверни голову в три четверти влево, открой ухо справа на изображении, больше шеи и плеча. Измени выражение лица и контур рта, прорисуй кольцевые узоры радужек.

Передай отметины на коже короткими чёрными штрихами и точками. Уменьши высокий воротник, обозначь складки одежды линиями. Замени фон чистым белым полем. Сохрани узнаваемые черты Саске.

## Качество кэпшенинга радикально влияет на сходимость

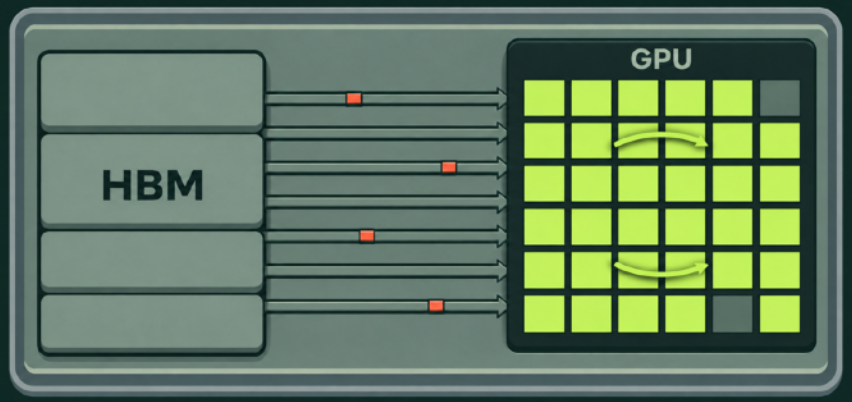
Пропуски и противоречия в инструкции создают шум в обучающем сигнале

# Кэпшенинг: GPU ждёт данные

MFU: полезные FLOP/с модели как доля вычислительного пика

## Compute bound

Оптимизированный претрейн



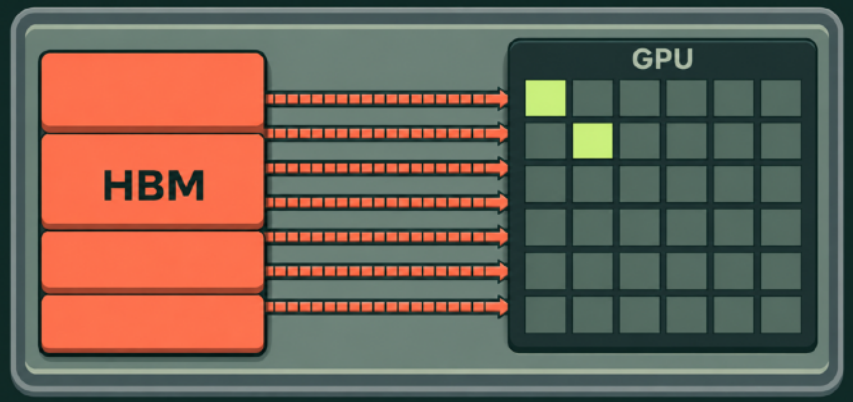

Ядра заняты: много токенов
на одну загрузку весов

**MFU 50–55%**

Ориентир оптимизированного обучения
Пример: Megatron, 54,2% MFU

## Memory bound

Одиночный decode, batch=1




Ядра ждут: на каждый новый токен
снова читаем веса из HBM

**MFU ≈0,34%***

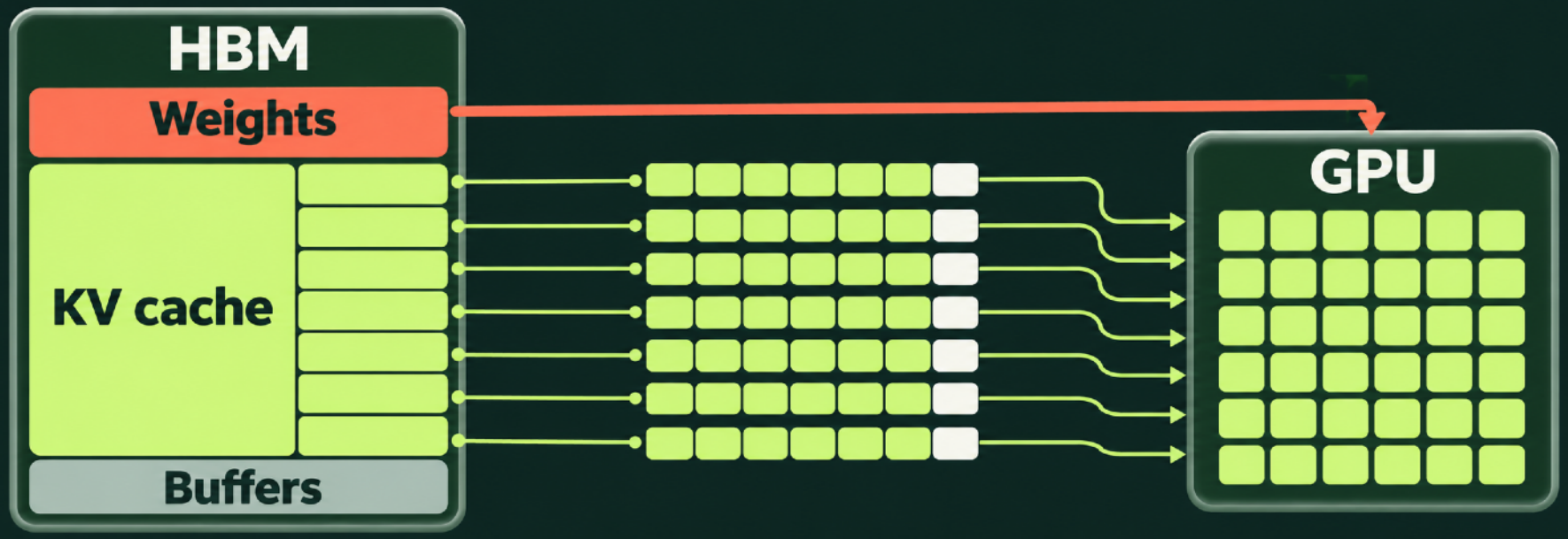
Оценка по весам: ≈1 FLOP/байт
H100 SXM, BF16. Не замер VLM

Prefill и vision encoder могут быть compute bound. 0,34% относится к decode, а не ко всей VLM

# Кэпшенинг: приоритет throughput

Цель: максимум инструкций на GPU·час
Latency отдельного запроса вторична

## Много независимых декодов, одни общие веса




Цель конфигурации: малая доля весов
Основная часть HBM под KV-кэш

Больше запросов в одном шаге
Переиспользуем веса, повышаем MFU

Рост до насыщения GPU. Нужен резерв под активации, vision encoder и рабочие буферы.

# Ускорение массового кэпшенинга



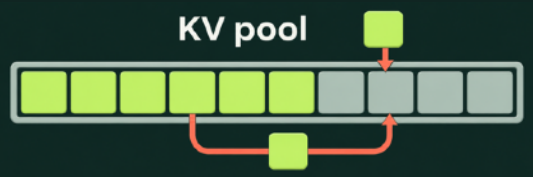
## Continuous batching

Завершённый запрос сразу заменяем новым
Батч остаётся заполненным




## FP8 на Hopper, NVFP4 на Blackwell

Меньше трафика и коммуникаций в низкой точности
Отдельно квантуем KV-кэш: меньше памяти на запрос*



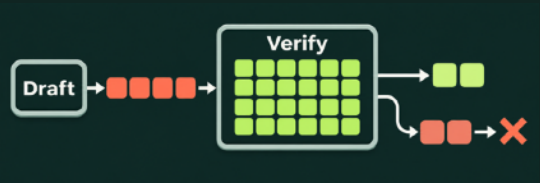

## Фиксированный пул KV-кэша

Заранее выделяем память и переиспользуем блоки
Избегаем переаллокаций на каждом шаге




## Компиляция, fusion и CUDA Graphs

Меньше CPU dispatch и пауз между GPU-ядрами
Особенно полезно при маленьком батче




## Без speculative decoding по умолчанию

При compute bound draft и проверка добавляют работу и могут снизить throughput. Решает замер.

* Форматы весов и KV задаём отдельно. Поддержка зависит от модели и backend. Проверяем качество.